



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 10 - SQL



WHAT IS SQL?

- Structured Query Language (SQL) is the standard language used to interact with relational databases.
 - It allows users to store, retrieve, update and manage data efficiently through simple commands.
 - It is known for its user-friendly syntax and powerful capabilities, SQL is widely used across industries.

Before SQL



- Manual file systems
- Slow retrieval
- Inconsistent storage

After SQL



- Structured data
- Fast querying
- Data integrity

WHAT IS SQL?

- We interact with databases using SQL queries. DBMS tools like MySQL and SQL Server have their own SQL engine and an interface where users can write and execute SQL queries.
- Below are the detailed steps involved in the SQL query execution.
 1. **Input:** The user submits a query (e.g., SELECT, INSERT, UPDATE, DELETE) via an application or interface.
 2. **Parsing:** The query processor breaks the query into parts (tokens) and checks for syntax and schema correctness.
 3. **Optimization:** The optimizer finds the most efficient way to run the query using indexes, statistics and available resources.
 4. **Execution:** The execution engine runs the query using the chosen plan, accessing or modifying the database as needed.
 5. **Output:** Results are returned to the user, either data (for SELECT) or a success message (for other operations).

WHAT IS SQL?

■ Key Components of a SQL System

- **Databases** :A database is a structured collection of data. It organizes data into **tables**, which are like spreadsheets with rows (records) and columns (fields) .
- **Tables**: Each table enforces rules and relationships among its columns for data integrity.
- **Indexes**: Indexes speed up queries by allowing the database to quickly locate data without scanning the entire table.
- **Views**:A view is a virtual table basically a saved SELECT statement you can query like a table.
- **Stored Procedures**:These are pre-written SQL scripts stored inside the database.They can receive inputs, run complex logic and return results boosting performance, reusability and security.
- **Transactions**:A transaction groups multiple SQL operations into a single unit. It ensures **all** changes are applied successfully or none are, preserving data integrity (ACID properties)
- **Security and Permissions**: SQL includes tools to **restrict access**, letting DBAs assign who can do what whether it's accessing tables, executing procedures, or changing structures.
- **Joins**: Joins combine data from multiple tables based on relationships essential for querying across related datasets.

WHAT IS SQL?

■ Rules for Writing SQL Queries

- **End with Semicolon (;):** Each SQL statement must end with a semicolon to execute properly.
- **Case Insensitivity:** SQL keywords (e.g., SELECT, INSERT) are not case-sensitive. However, table or column names may be case-sensitive depending on the DBMS.
- **Whitespace Allowed:** Queries can span multiple lines, but use spaces between keywords and names.
- **Reserved Words:** Avoid using SQL keywords as names. If needed, wrap them in quotes (" ") or backticks (`).
- **Comments**
 - ❑ Single-line: -- comment
 - ❑ Multi-line: /* comment */
- **Data Constraints:** Use NOT NULL, UNIQUE, PRIMARY KEY, etc., to ensure data accuracy.
- **String Values:** Enclose strings in single quotes ('text').
- **Naming Rules:**
 - ❑ We Start with a letter
 - ❑ We can only use Max 30 characters
 - ❑ We only use letters, numbers and underscores (_)

WHAT IS SQL?

Benefits of SQL

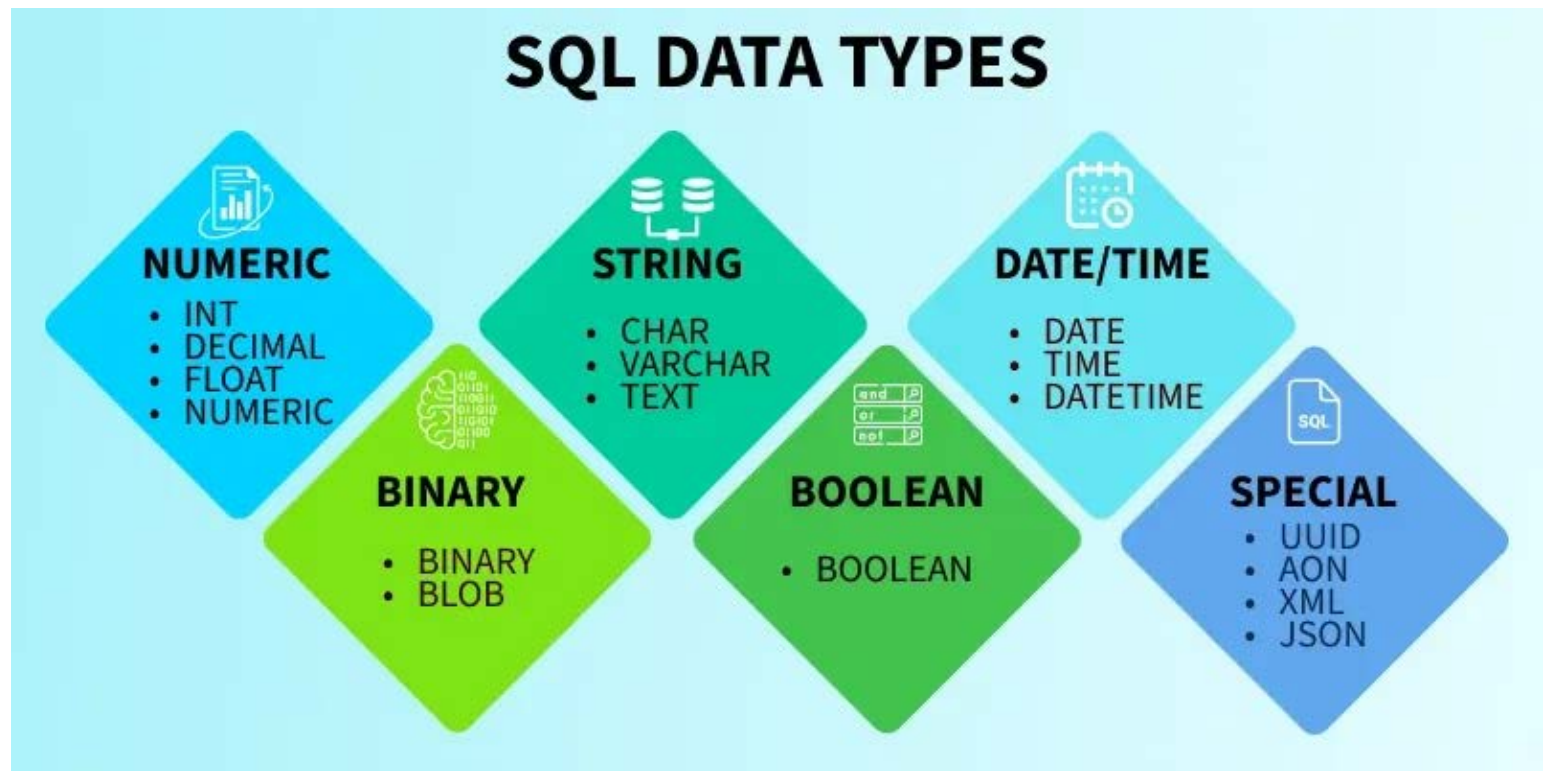
- **Efficiency:** SQL is designed to handle complex queries and large datasets with optimal performance, making data retrieval and manipulation seamless.
- **Standardization:** As an ANSI and ISO standard language, SQL provides a universal method to interact with relational databases across platforms.
- **Scalability:** SQL supports databases ranging from small-scale applications to enterprise-level systems, ensuring smooth operations regardless of size.
- **Flexibility:** SQL can be extended with procedural programming (e.g., PL/SQL, T-SQL) to build complex business logic and custom functions.

Limitations of SQL

- **Complexity in Advanced Operations:** Advanced functionalities such as indexing, query optimization and performance tuning require in-depth technical knowledge.
- **Scalability Concerns:** SQL performs best with structured data but handling unstructured data or massive distributed systems can pose challenges.
- **Platform-Specific Variations:** While SQL is standardized, many databases implement unique extensions, leading to portability and compatibility issues.
- **Lack of Real Time Analytics:** Traditional SQL databases are not optimized for real-time data ingestion and analysis.

SQL DATA TYPES

- In SQL, each column must be assigned a data type that defines the kind of data it can store, such as integers, dates, text, or binary values. Choosing the correct data type is crucial for data integrity, query performance and efficient indexing.



SQL DATA TYPES

I. Numeric Data Types

- Numeric data types are fundamental to database design and are used to store numbers, whether they are integers, decimals or floating-point numbers. These data types allow for mathematical operations like addition, subtraction, multiplication and division, which makes them essential for managing financial, scientific and analytical data.
- **Exact Numeric Datatype**
- Exact numeric types are used when precise numeric values are needed, such as for financial data, quantities, and counts. Some common exact numeric types include:

Data Types	Description	Range
BIGINT	Large integer numbers.	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
INT	Standard integer values.	-2,147,483,648 to 2,147,483,647
SMALLINT	Small integers.	-32,768 to 32,767
TINYINT	Very small integers.	0 to 255
DECIMAL	Exact fixed-point numbers (e.g., for financial values).	$-10^{38} + 1$ to $10^{38} - 1$
NUMERIC	Similar to DECIMAL, used for precision data.	$-10^{38} + 1$ to $10^{38} - 1$

SQL DATA TYPES

- **Approximate Numeric Datatype**
- These types are used to store approximate values, such as scientific measurements or large ranges of data that don't need exact precision.

Data Type	Description	Range
FLOAT	Approximate numeric values.	-1.79E+308 to 1.79E+308
REAL	Similar to FLOAT, but with less precision.	-3.40E+38 to 3.40E+38

SQL DATA TYPES

2. Character and String Data Types

- Character data types are used to store text or character-based data. The choice between fixed-length and variable-length data types depends on the nature of your data.

Data Types	Description
CHAR	With a maximum length of 8000 characters. (Fixed-Length non-Unicode Characters)
VARCHAR	With a maximum length of 8000 characters. (Variable-Length non-Unicode Characters)
TEXT	With a maximum length of 2,127,483,647 characters. (Variable Length non-Unicode data)

SQL DATA TYPES

3. Date and Time Data Type

- SQL provides several data types for storing date and time information. They are essential for managing timestamps, events and time-based queries. These are given in the below table.

Data Types	Description	Storage Size
DATE	Stores the date (year, month, day).	3 Bytes
TIME	Stores the time (hour, minute, second).	3 Bytes
DATETIME	Store both the date and time (year, month, day, hour, minute, second).	8 Bytes

SQL DATA TYPES

4. Binary Data Types in SQL

- Binary data types are used to store binary data such as images, videos or other file types. These include:

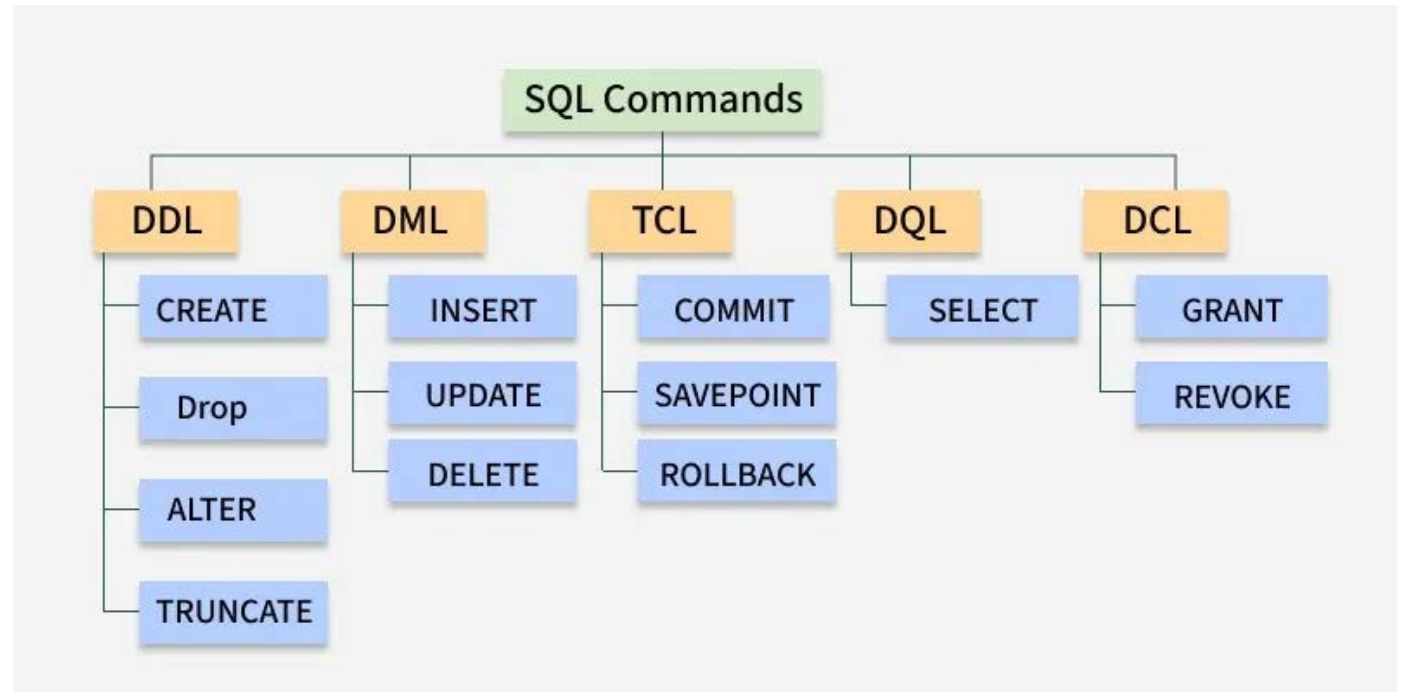
Data Types	Description	Max Length
BINARY	Fixed-length binary data.	8000 bytes
VARBINARY	Variable-length binary data.	8000 bytes
IMAGE	Stores binary data as images.	2,147,483,647 bytes

5. Boolean Data Type in SQL

- The BOOLEAN data types are used to store logical values, typically TRUE or FALSE. It is commonly used for flag fields or binary conditions.

SQL COMMANDS

- Structured Query Language (SQL) commands are standardized instructions used by developers to interact with data stored in **relational databases**. These commands allow for the creation, manipulation, retrieval and control of data, as well as database structures. SQL commands are categorized based on their specific functionalities.



SQL COMMANDS

I. DDL - Data Definition Language

- DDL (Data Definition Language) consists of SQL commands that can be used for defining, altering and deleting database structures such as tables, indexes and schemas.

Command	Description	Syntax
CREATE	Creates a database or its objects (table, index, functions, views, store procedure and triggers).	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Deletes objects from the database.	DROP TABLE table_name;
ALTER	Alters the structure of the database.	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Removes all records from a table but keeps the table structure intact.	TRUNCATE TABLE table_name;
COMMENT	Adds comments to the data dictionary.	COMMENT ON TABLE table_name IS 'comment_text';
RENAME	Changes the name of an existing database object, such as a table.	RENAME TABLE old_table_name TO new_table_name;

SQL COMMANDS

2. DQL - Data Query Language

- DQL is used to fetch data from the database. The main command is SELECT, which retrieves records based on the query. The output is returned as a result set (a temporary table) that can be viewed or used in applications.

Command	Description	Syntax
SELECT	Retrieves data from the database.	SELECT column1, column2, ...FROM table_name WHERE condition;
FROM	Indicates the table(s) from which to retrieve data.	SELECT column1 FROM table_name;
WHERE	Filters rows before any grouping or aggregation.	SELECT column1 FROM table_name WHERE condition;
GROUP BY	Groups rows that have the same values in specified columns.	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1;
HAVING	Filters the results of GROUP BY.	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1 HAVING condition;
DISTINCT	Removes duplicate rows from the result set.	SELECT DISTINCT column1, column2, ... FROM table_name;
ORDER BY	Sorts the result set by one or more columns.	SELECT column1 FROM table_name ORDER BY column1 [ASC DESC];
LIMIT	By default, it sorts in ascending order unless specified as DESC.	SELECT * FROM table_name LIMIT number;

SQL COMMANDS

3. Data Manipulation Language

- DML commands are used to manipulate the data stored in database tables. With DML, you can insert new records, update existing ones, delete unwanted data or retrieve information.

Command	Description	Syntax
INSERT	Inserts data into a table.	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Updates existing data within a table.	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Deletes records from a table.	DELETE FROM table_name WHERE condition;

SQL COMMANDS

4. Data Control language

- DCL (Data Control Language) includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions and other controls of the database system. These commands are used to control access to data in the database by granting or revoking permissions.

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

SQL COMMANDS

I. Transaction Control Language

- Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, transaction fails. Therefore, a transaction has only two results: success or failure.

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction.	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the current transaction on a permanent basis. Some databases provide an auto-commit feature, which can be configured using settings.	COMMIT;
ROLLBACK	Reverts changes made during the current transaction, ensuring no unwanted changes are saved.	ROLLBACK;
SAVEPOINT	Sets a point within a transaction to which changes can be rolled back, allowing partial rollbacks	SAVEPOINT savepoint_name;

SQL CREATE DATABASE

- CREATE command in SQL is used to define new databases and tables. A database acts as a container that holds related objects such as tables, views and procedures, while a table stores data in rows and columns.
- To create a new database in SQL, we use CREATE DATABASE command followed by the database name. Database names cannot contain spaces; if needed, an underscore (_) can be used instead.
- Let's create a simple database called University:

```
CREATE DATABASE University;
```
- If you try to create a database with a name that already exists, you'll see an error. To avoid this, either choose a new name or use IF NOT EXISTS clause to only create database if it doesn't already exist.

```
CREATE DATABASE IF NOT EXISTS University;
```
- **Explanation:** This query creates an empty database named University. A success message will confirm that the database was created.

SQL CREATE DATABASE

1. Verifying Database Creation

- To confirm that a new database has been created, use:

```
SHOW DATABASES;
```

- **Explanation:** This command lists all databases in the system. From this list, you can check if University has been added.

2. Switching to a Database (USE Command)

- Once your database is created, we can switch to that database to begin adding tables, inserting data and performing queries. To switch to your new database, use the USE command.

```
USE University;
```

- **Explanation:** After running this, any SQL operations like creating tables or inserting data will be done within **University** database. This allows you to work within the scope of the newly created database.

3. Deleting a Database

- If you ever want to delete a database, use:

```
DROP DATABASE University;
```

- This permanently deletes the database.

SQL CREATE TABLE

- A table is the core structure of a database where data is stored in rows and columns. While creating a table, you must define:
- Column names
- Data types (e.g., INT, VARCHAR, DATE)
- Constraints (e.g., PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, FOREIGN KEY)

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    ...  
);
```

SQL CREATE TABLE

- **Example: Creating a Students Table**

- Here, we are creating a table named Students to store student details like ID, name, subject, year and marks.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Subject VARCHAR(50),  
    Year INT,  
    Mark INT  
);
```

- **Explanation:** In this table, StudentID is the unique identifier (Primary Key), Name must not be empty, Subject stores the subject name, Year represents the study year and Mark stores the mark scored by the student.
- If you attempt to create a table that already exists, SQL will throw an error. To avoid this, you can use the IF NOT EXISTS clause.

```
CREATE TABLE IF NOT EXISTS Students (...);
```

SQL CREATE TABLE

- **PRIMARY KEY in SQL** is a column (or group of columns) that uniquely identifies the records in that table. A primary key must contain **unique values** and can not have any **NULL value**.
- **FOREIGN KEY** is a column or set of columns in one table that references the primary key columns of another table. This creates a **relationship** between the two tables, ensuring that the child table (which contains the foreign key) can only have values that exist in the parent table's primary key column(s).

- **Example:**

```
CREATE TABLE Courses(  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50) NOT NULL,  
    Instructor VARCHAR(50),  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
);
```

- The NOT NULL constraint in SQL ensures a column must always contain a value and cannot be left empty.

SQL CREATE TABLE

1. Verifying Table Creation

- After running the CREATE TABLE query, the system shows a success message like Table created successfully. To confirm that the table exists, we can list all tables in the current database using:

```
SHOW TABLES;
```

- This will display all the tables created inside the database and from here you can confirm that Students table has been successfully created.

2. Deleting a Table

- If you no longer need a table, use:

```
DROP TABLE Students;
```

- **Explanation:** This removes the Students table permanently from the database.

SQL CREATE TABLE

- **Create Table from Existing Table**

- We can also create a new table based on the structure (and optionally the data) of an existing table. The `CREATE TABLE AS SELECT` command allows us to duplicate an entire table or select specific columns to form a new one.

Syntax:

```
CREATE TABLE new_table_name AS  
SELECT column1, column2, ...  
FROM existing_table_name  
WHERE ...;
```

- In this example, we create a new table `SubTable` that contains just the `StudentID` and `Name` columns from the `Students` table. This method is useful for creating backups or performing quick data migrations.

Query:

```
CREATE TABLE SubTable AS  
SELECT StudentID, Name  
FROM Students;
```

SQL ALTER TABLE

- SQL ALTER TABLE statement modify the structure of an existing table in a database. Whether adding new columns, modifying existing ones, deleting columns or renaming them. ALTER TABLE statement enables you to make changes without losing data stored in the table.

Syntax:

```
ALTER TABLE table_name [ADD | DROP | MODIFY] column_name datatype;
```

- **Common Use Cases for SQL ALTER TABLE**

I. **ADD**

- The ADD clause is used to add a new column to an existing table. You must specify the name of the new column and its data type.

Syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

- **Example:**

```
ALTER TABLE Students  
ADD Email varchar(255);
```

- Here, we are adding a column named **Email** to **Students** table.

SQL ALTER TABLE

2. MODIFY

- The MODIFY clause is used to modify the definition of an existing column, such as changing its data type or size.

Syntax:

```
ALTER TABLE table_name  
MODIFY COLUMN column_name datatype;
```

■ Example:

```
ALTER TABLE Students  
MODIFY COLUMN Address VARCHAR(100);
```

- Here, we are modifying the column named **Address** datatype that is **VARCHAR(100)**.

SQL ALTER TABLE

3. DROP

- The DROP clause allows you to remove a column from a table. Be cautious when using this command as it will permanently remove the column and its data.

Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

■ Example:

```
ALTER TABLE Students  
DROP COLUMN Mark;
```

- Here, we are removing a column named **Mark** from **Students** table.

SQL ALTER TABLE

4. RENAME COLUMN

- We can rename an existing column using RENAME COLUMN clause. This allows you to change the name of a column while preserving its data type and content.

Syntax:

```
ALTER TABLE table_name  
RENAME COLUMN old_name TO new_name;
```

■ Example:

```
ALTER TABLE Students  
RENAME COLUMN Name TO LastName;
```

- This renames the column **Name** to **LastName** in the Customer table.

SQL ALTER TABLE

- **RENAME TO**

- We can rename an entire table using the RENAME TO clause. This changes the name of the table while preserving its structure and data.

Syntax:

```
ALTER TABLE table_name  
RENAME TO new_table_name;
```

- **Example:**

```
ALTER TABLE Students  
RENAME TO UniStudents;
```

- This renames the table from **Students** to UniStudents.

SQL QUERIES

- The SQL SELECT statement retrieves data from one or more tables and returns it as a tabular result set of rows and columns. You can fetch all columns with * or choose specific columns, and narrow results using WHERE filters. Queries can be ordered with ORDER BY, grouped and aggregated with GROUP BY/HAVING, and combined across tables using JOINS.

Syntax:

```
SELECT column1, column2.... FROM table_name;
```

- column1, column2: columns you want to retrieve.
- table_name: name of the table you're querying.

■ **Example 1: Select Specific Columns**

In this example, we will demonstrate how to retrieve specific columns from the Students table. Here we will fetch only Name and Year for each record.

```
SELECT Name, Year  
FROM Students;
```

■ **Example 2: Select All Columns**

- In this example, we will fetch all the fields from table Students:

```
SELECT * FROM Students;
```

SQL QUERIES

- The SQL INSERT INTO statement is used to add new records into a table. It allows inserting data into all columns or specific ones, depending on the requirement.
 - Can insert a single row or multiple rows at once.
 - Supports inserting data directly or from another table using a subquery.

I. Inserting Data into All Columns

- This method is used when you want to insert data into all columns of a table without specifying column names. We simply provide the values for each column, in the same order that the columns are defined in the table.

Syntax:

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

- **table_name:** name of the table where data will be inserted
- **value1, value2... :** values that correspond to each column in order.
- **Example:** Let's see how the INSERT INTO statement works with a practical examples.

```
INSERT INTO Students  
VALUES (159, 'Liam', 'Physics', 3, 8);
```


SQL QUERIES

2. Inserting Data into Specific Columns

- In some cases, you might want to insert data into only certain columns, leaving the others empty or with default values. In such cases, we can specify the column names explicitly.

Syntax

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

■ Parameters:

- **table_name:** name of the table.
- **column1, column2..:** name of first column, second column.
- **value1, value2, value3..:** the values for each specified column of the new record.

- **Example:** Let's say we only want to insert the student's ID, name, and year into the Students table.

```
INSERT INTO Students (StudentID, Name, Year)
VALUES (106, 'Hiroshi', 1);
```

SQL QUERIES

3. Inserting Multiple Rows at Once

- Instead of running multiple INSERT INTO commands, you can insert multiple rows into a table in a single query. This is more efficient and reduces the number of database operations.

Syntax

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...), (value1, value2, ...), (value1, value2, ...);
```

- **Example:** If we want to add multiple students to Students table in one go, query would look like this:

```
INSERT INTO Students (StudentID, Name, Subject, Year, Mark)  
VALUES  
(70, 'Mateo Garcia', 'Programming', 2, 7),  
(82, 'Hana Suzuki', 'Web Design', 4, 10),  
(94, 'Oliver Jensen', 'Digital Signal Processing', 3, 8),  
(107, 'Amelia Brown', 'Algebra and Statistics', 2, 9);
```

SQL QUERIES

4. Inserting Data from One Table into Another Table

- We can also copy data from one table into another table using the INSERT INTO SELECT statement. This is very useful when we want to move or replicate data from one table to another without manually typing all the data.

- **Method 1: Insert All Columns from Another Table**

- Inserts every column from source table into destination table.

```
INSERT INTO target_table  
SELECT * FROM source_table;
```

- **Method 2: Insert Specific Columns from Another Table**

- Allows inserting only selected columns from the source table.

```
INSERT INTO target_table (col1, col2, ...)  
SELECT col1, col2, ...  
FROM source_table;
```

SQL QUERIES

- The SQL UPDATE statement is used to modify existing records in a table. It allows you to change one or more column values for specific rows using the WHERE clause. Without a WHERE condition, all rows in the table will be updated.

Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition
```

- **table_name:** Name of the table you want to update.
- **SET:** The column(s) you want to update and their new values.
- **WHERE:** Filters the specific rows you want to update.
- The SET keyword assigns new values to columns, while the WHERE clause selects which rows to update. Without WHERE, all rows will be updated.

Example:

```
UPDATE Students  
SET Name = 'John Smith',  
Year = 4  
WHERE StudentID = 94;
```

SQL QUERIES

- The SQL DELETE statement is used to remove specific rows from a table while keeping the table structure intact. It is different from DROP, which deletes the entire table.
 - It removes rows based on conditions.
 - Retains table schema, constraints, and indexes.
 - Can delete a single row or all rows.

Syntax:

```
DELETE FROM table_name  
WHERE some_condition;
```

- **Some_condition:** A condition used to filter the rows you want to delete.
- **table_name:** The name of the table from which you want to delete the rows.
- We can delete single or multiple records using the WHERE clause; if it's omitted, all records in the table are removed.
- **Example:**

```
DELETE FROM Students  
WHERE StudentID = 159;
```

SQL CLAUSES

- The SQL WHERE clause filters rows based on one or more conditions, so your query returns (or modifies) only the records that match. It's used across SELECT, UPDATE, and DELETE statements, and works with data from a single table or from multiple tables after joins.

Syntax:

```
SELECT column1, column2  
FROM table_name  
WHERE column_name operator value;
```

- **column1, column2:** Columns you want to retrieve.
- **table_name:** Table you are querying from.
- **operator:** Comparison logic (e.g., =, <, >, LIKE).
- **value:** The value or pattern to filter against.
- **Example:**

```
SELECT Name, Subject, Mark  
FROM Students  
WHERE Mark >= 5;
```

SQL CLAUSES

- The SQL GROUP BY clause is used to arrange identical data into groups based on one or more columns. It is commonly used with aggregate functions like COUNT(), SUM(), AVG(), MAX(), and MIN() to perform calculations on each group of data.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1, column2;
```

- **aggregate_function:** function used for aggregation, e.g., SUM(), AVG(), COUNT().
- **table_name:** name of the table from which data is selected.
- **condition:** Optional condition to filter rows before grouping (used with WHERE).
- **column1, column2:** Columns on which the grouping is applied.
- **Example:** Using GROUP BY with multiple columns groups rows that share the same values in those columns. For example, grouping by subject and year will combine rows with the same subject–year pair and we can count how many students fall into each group.

```
SELECT Subject, Year, COUNT(*)
FROM Students
GROUP BY Subject, Year;
```

SQL CLAUSES

- The SQL HAVING clause filters the results of grouped data after using the GROUP BY clause. It is used with aggregate functions such as SUM(), COUNT(), or AVG() to display only those groups that meet specific conditions.

Syntax:

```
SELECT AGGREGATE_FUNCTION(column_name)
FROM table_name
HAVING condition;
```

- In this example, we calculate the average mark of all students and display it only if the average is equal or greater than 5.

- **Example:**

```
SELECT AVG(Mark) AS AverageMark
FROM Students
GROUP BY Subject
HAVING AVG(Mark) >= 5;
```


SQL CLAUSES

- The ORDER BY clause in SQL is used to sort query results based on one or more columns in either ascending (ASC) or descending (DESC) order. By default, it sorts in ascending order (lowest to highest).
- To sort in descending order, use the DESC keyword.

Syntax:

```
SELECT * FROM table_name ORDER BY column_name ASC | DESC;
```

- **table_name**: name of the table.
- **column_name**: name of the column according to which the data is needed to be arranged.
- **ASC**: to sort the data in ascending order.
- **DESC**: to sort the data in descending order.
- In the next example, we will fetch all data from the table Students and then sort the result in descending order first, according to the column Name, and then in ascending order, according to the column Year.
- To sort according to multiple columns, separate the names of columns by the (,) operator.
- **Example:**

```
SELECT * FROM Students ORDER BY Name DESC, Year ASC;
```

SQL OPERATORS

SQL Operators

Arithmetic Operators

+, -, *, /, %

Comparison Operators

=, !=, <>, >, <, >=, <=

Logical Operators

AND, OR, NOT

Bitwise operator

&, |, ^, ~, <<, >>

Compound Operator

+=, -=, *=, /=, %=

Special Operators

BETWEEN ... AND ..., IN(...),
LIKE, IS NULL, EXISTS

- SQL Logical Operators are used to test conditions in queries, returning results as TRUE, FALSE, or UNKNOWN. They help in combining, negating, and comparing conditions, enabling precise data retrieval and filtering.
 - Combine multiple conditions in a query.
 - Control query execution flow.
 - Support complex filtering and comparison.
 - Return boolean results for decision-making.

SQL OPERATORS

1. AND Operator

- The AND operator is used to combine two or more conditions in an SQL query. It returns records only when all conditions specified in the query are true. This operator is commonly used when filtering data that must satisfy multiple criteria simultaneously.

- **Example:**

```
SELECT *  
FROM Students  
WHERE Subject = 'Web Programming' AND Mark = 10;
```

2. IN Operator

- The IN operator checks whether a value matches any value from a specified list, making queries more concise and readable than using multiple OR conditions. It is especially useful for filtering results by several possible values in a column, helping reduce query complexity.

- **Example:**

```
SELECT *  
FROM Students  
WHERE Subject IN ('Embedded Systems', 'Complexity and Computability');
```

SQL OPERATORS

3. NOT Operator

- The NOT operator is used to reverse the result of a condition, returning TRUE when the condition is FALSE. It is typically used to exclude records that match a specific condition, making it useful for filtering out unwanted data.
- **Example:** Retrieve the records of students from the Students table whose names do not start with the letter 'S'.

```
SELECT *  
FROM Students  
WHERE Name NOT LIKE 'S%';
```

4. OR Operator

- The OR operator combines multiple conditions in a SQL query and returns TRUE if at least one of the conditions is satisfied. It is ideal for situations where you want to retrieve records that meet any of several possible conditions.
- **Example:**

```
SELECT *  
FROM Students  
WHERE Subject = 'Artificial Intelligence' OR Subject = 'Web Programming';
```

SQL OPERATORS

5. LIKE Operator

- The LIKE operator in SQL is used in the WHERE clause to search for a specified pattern in a column. It is particularly useful when we want to perform pattern matching on string data.
- Wildcards are used with the LIKE operator to search for specific patterns in strings. They substitute one or more characters in the string. There are four wildcard characters in SQL:
 - **% (Percent)**: Represents zero or more characters.
 - **_ (Underscore)**: Represents a single character.
 - **[] (Square Brackets)**: Represents any single character within brackets.
 - **- (Hyphen)**: Specify a range of characters inside brackets.
- **Example:** Retrieve the records of students from the Students table whose names start with the letter 'M'.

```
SELECT *  
FROM Students  
WHERE Name LIKE 'M%';
```

SQL OPERATORS

Pattern	Meaning
'a%'	Match strings that start with 'a'.
'%a'	Match strings that end with 'a'.
'a%t'	Match strings that start with 'a' and end with 't'.
'%wow%'	Match strings that contain the substring 'wow' at any position.
'_wow%'	Match strings that contain the substring 'wow' at the second position.
'_a%'	Match strings that contain 'a' at the second position.
'a__%'	Match strings that start with 'a' and contain at least 2 more characters.

SQL OPERATORS

6. BETWEEN Operator

- The BETWEEN operator in SQL allows us to test if a value or expression lies within a specified range.
 - Includes both the lower and upper limits in the result set.
 - Works with numbers, dates, and text values.
 - Ideal for selecting data within a defined range.

- **Example:**

```
SELECT *  
FROM Students  
WHERE Mark BETWEEN 5 AND 8;
```

7. IS NULL Operator

- The SQL IS NULL operator checks whether a column contains a NULL (missing or unknown) value. Since NULL is not the same as zero or an empty string, IS NULL is used in the WHERE clause to filter rows where a value is absent.

- **Example:**

```
SELECT * FROM Students  
WHERE Mark IS NULL;
```

SQL AGGREGATE FUNCTIONS

I. COUNT()

- The SQL COUNT() function is used to return the number of rows or non-null values in a column. It helps summarize data in reports and analytics.
 - Counts all rows or rows with a condition.
 - Counts unique values using COUNT(DISTINCT column).
 - Commonly used with GROUP BY and HAVING for grouped results.

■ Example:

```
SELECT COUNT(*) AS TotalStudents FROM Students;  
SELECT COUNT(DISTINCT Subjects) AS UniqueSubjects  
FROM Students;
```


SQL AGGREGATE FUNCTIONS

2. SUM()

- The SQL SUM() function is an aggregate function used to calculate the total value of a numeric column. It is widely used in reporting, financial calculations, and data analysis to quickly get overall totals.
 - Returns a single total of selected numeric values.
 - Works with WHERE, GROUP BY, and HAVING for conditional sums.
 - Supports expressions and DISTINCT to sum unique values only.

- **Example:**

```
SELECT SUM(Mark) AS TotalMarks  
FROM Students;
```

SQL AGGREGATE FUNCTIONS

3. **AVG()**

- The **AVG()** function in SQL calculates the average of a numeric column. It helps identify the central value of data by ignoring NULL entries.
 - Finds the average or mean using only non-null values.
 - Useful for quick comparisons and basic data analysis.

■ **Example:**

```
SELECT AVG(Mark) AS AverageMarks  
FROM Students;
```

SQL AGGREGATE FUNCTIONS

4. MIN()

- The MIN() function in SQL returns the smallest value from a specified column or expression. It is used to identify minimum numeric, date, or string values while automatically ignoring NULL values.

- Works with WHERE, GROUP BY, HAVING, and subqueries.

- **Example:**

```
SELECT MIN(Mark) AS LowestMark  
FROM Students;
```

5. MAX()

- The SQL MAX() function helps quickly identify the highest value within any column. It is widely used in reports to find top values like peak sales, latest dates, or highest scores.

- Supports numeric, date, and text data types.

- Skips NULL values during calculation.

- **Example:**

```
SELECT MAX(Mark) AS HighestMark  
FROM Students;
```

SQL JOINS

- SQL joins are fundamental tools for combining data from multiple tables in relational databases.
 - For example, consider two tables where one table (say Students) has student information with id as a key and other table (say Marks) has information about marks of every student id. Now to display the marks of every student with name, we need to join the two tables.
 - Please remember, we store data into multiple tables as part of database normalization to avoid anomalies and redundancies.

I. SQL INNER JOIN

- The INNER JOIN keyword selects all rows from both tables as long as the condition is satisfied.

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1, ... FROM table1 INNER JOIN  
table2 ON table1.matching_column = table2.matching_column;
```

- **Note:** We can also write JOIN instead of INNER JOIN. JOIN is the same as INNER JOIN.
- **Example:**

```
SELECT Courses.CourseID, Students.Name, Students.Year FROM Students  
INNER JOIN Courses  
ON Students.StudentID = Courses.StudentID;
```

SQL JOINS

2. SQL LEFT JOIN

- LEFT JOIN returns all the rows of the table on the left side of the join and matching rows for the table on the right side of the join. If there is no match, NULL values are returned for columns from the right table. LEFT JOIN is also known as LEFT OUTER JOIN.

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1, ....  
FROM table1  
LEFT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

- **Example:**

```
SELECT Students.Name, Courses.CourseID  
FROM Students  
LEFT JOIN Courses  
ON Courses.StudentID = Students.StudentID;
```

SQL JOINS

3. SQL RIGHT JOIN

- RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. If there is no match, NULL values are returned for columns from the left table. RIGHT JOIN is also known as RIGHT OUTER JOIN.

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1, ....  
FROM table1  
RIGHT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

■ Example:

```
SELECT Students.Name, Courses.CourseID  
FROM Students  
RIGHT JOIN Courses  
ON Courses.StudentID = Students.StudentID;
```

SQL JOINS

4. SQL FULL JOIN

- FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1, ....  
FROM table1  
FULL JOIN table2  
ON table1.matching_column = table2.matching_column;
```

- **Example:**

```
SELECT Students.Name, Courses.CourseID  
FROM Students  
FULL JOIN Courses  
ON Courses.StudentID = Students.StudentID;
```